



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica

CORSO DI LAUREA IN INFORMATICA

Insegnamento: Sviluppo Web Avanzato

Progetto Foody API (WebDelivery)

Gasparroni Maikol

291909

Indice

1 Introduzione	4
2 Sviluppo del Database	4
2.1 Progettazione concettuale	4
2.1.1 Modello E-R Iniziale	4
2.1.2 Descrizione del Diagramma E-R iniziale	5
2.1.3 Elementi non visibili nella figura e scelte progettuali	5
2.1.4 Spiegazione delle Cardinalità	5
2.1.5 Formalizzazione dei vincoli non esprimibili nel modello ER	7
2.2 Ristrutturazione dello schema E-R	7
2.2.1 Analisi delle ridondanze	7
2.2.2 Eliminazione della gerarchia Utente	8
2.2.3 Introduzione di identificatori surrogati e ridenominazione	8
2.3 Traduzione del modello E-R nel modello relazionale	8
3 Scelte Progettuali	9
3.1 Paradigma Collection-Item	9
3.2 Design Pattern DAO	9
3.3 Sicurezza tramite JSON Web Token	9
3.4 Gestione delle Transazioni	9
3.5 Gestione Globale delle Eccezioni	9
3.6 Cross-Origin Resource Sharing	10
4 Dipendenze Software	10
5 Funzionalità Realizzate	10
5.1 Gestione Autenticazione	10
5.2 Gestione Catalogo	10
5.3 Gestione Ordini	10
1.5.4 Data Masking	10
6 Funzionalità Extra e Opzionali Realizzate	10
6.1 Gestione Dinamica del Catalogo	11
6.2 Dizionario Globale degli Ingredienti	11
6.3 Gruppi di Mutua Esclusione e Caratteristiche	12
6.4 Registrazione Utenti e Gestione Profilo	12
6.5 Gestione Dinamica del Personale	12
6.6 Data Masking e Sicurezza	12
7 Specifica dell'API	13
7.1 Autenticazione	13
7.2 Catalogo Prodotti	13
7.3 Ingredienti	15
7.4 Caratteristiche e Gruppi di Mutua Esclusione	15
7.5 Ordini	16
7.6 Personale	18
7.7 Clienti	18

Documentazione Progetto RESTful API - Foody

1 Introduzione

Il presente documento descrive l'architettura, le scelte progettuali e le specifiche dell'API RESTful "Foody", un sistema software per la gestione delle ordinazioni e delle consegne a domicilio. L'applicativo espone i propri servizi tramite interfaccia web, permettendo la gestione di utenti, prodotti, caratteristiche del menu e l'intero ciclo di vita degli ordini.

2 Sviluppo del Database

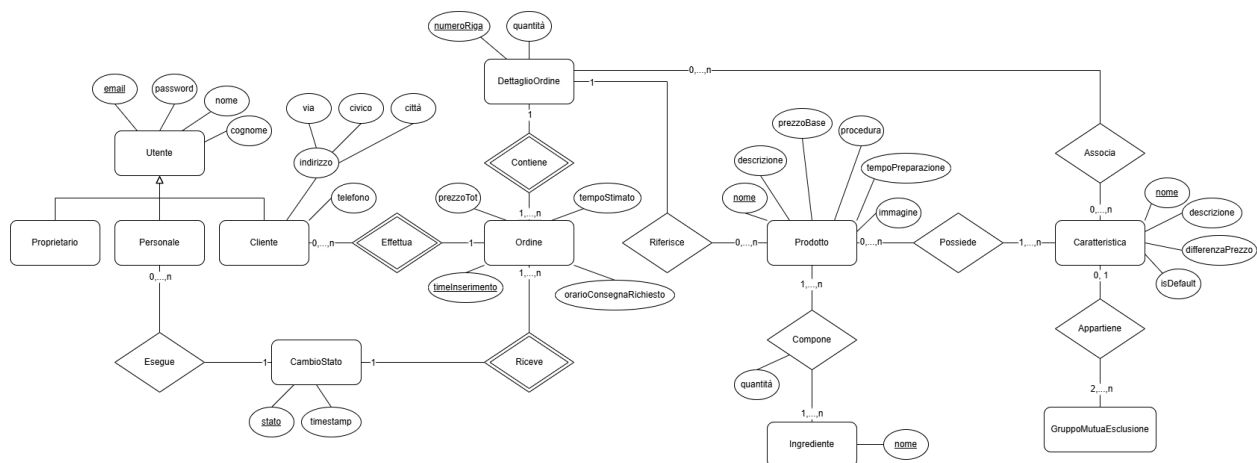
Il database costituisce l'infrastruttura centrale per la persistenza dei dati, garantendo la coerenza tra ordini, catalogo prodotti e anagrafica utenti attraverso un'architettura relazionale normalizzata.

2.1 Progettazione concettuale

In questa fase viene creato un primo schema della base di dati che possa modellare le entità e la complessità del dominio senza tenere conto di normalizzazione e ottimizzazione.

2.1.1 Modello E-R Iniziale

La seguente figura illustra il diagramma E-R e l'architettura inizialmente abbozzati per il data base alla base del quale l'API restful prende i suoi dati.



L'immagine originale è disponibile come allegato.

2.1.2 Descrizione del Diagramma E-R iniziale

Il diagramma E-R rappresenta la base di dati concettuale per il servizio di gestione delle ordinazioni e delle consegne a domicilio dell'attività di ristorazione "WebDelivery". Le entità principali incluse nel modello sono: Utente (con le sue specializzazioni Proprietario, Personale, Cliente), Ordine, DettaglioOrdine, Prodotto, Caratteristica, GruppoMutuaEsclusione, Ingrediente e CambioStato.

2.1.3 Elementi non visibili nella figura e scelte progettuali

Identificazione di DettaglioOrdine

L'entità DettaglioOrdine è stata modellata come un'entità debole. Essa è identificata parzialmente dall'attributo numeroRiga e legata in modo identificante all'entità Ordine tramite la relazione Contiene. Il legame con l'entità Prodotto tramite la relazione Riferisce, è invece non identificante. Questa scelta permette a un cliente di inserire lo stesso identico prodotto in più righe distinte dello stesso ordine, associandovi configurazioni di caratteristiche differenti, ad esempio due caffè in righe separate, uno zuccherato e uno amaro, evitando conflitti di chiavi duplicate.

Indirizzo del Cliente

L'attributo indirizzo dell'entità Cliente è modellato come attributo composto (via, civico, città). Questo livello di scomposizione è necessario a livello concettuale per supportare le successive operazioni applicative e i controlli sulla stringa di recapito per la consegna a domicilio.

Struttura del Tracciamento Stati

Per soddisfare il requisito di memorizzazione dello storico delle modifiche di stato di un ordine, si è introdotta l'entità debole CambioStato. Essa dipende dall'ordine di riferimento e dal timestamp dell'evento, legandosi dinamicamente al membro del Personale che esegue l'operazione.

2.1.4 Spiegazione delle Cardinalità

Effettua: Cliente (0,n) – Ordine (1,1)

Un cliente registrato può non aver ancora effettuato ordini nel sistema, ma ogni ordine deve appartenere a uno e un solo cliente specifico.

Contiene: Ordine (1,n) – DettaglioOrdine (1,1)

Un ordine deve contenere almeno una riga di dettaglio per esistere; ogni riga di dettaglio appartiene rigidamente a un solo ordine parent.

Riferisce: DettaglioOrdine (1,1) – Prodotto (0,n)

Ogni riga di dettaglio deve puntare a un preciso prodotto del menu; un prodotto può comparire in zero o molte righe d'ordine complessive.

Associa: DettaglioOrdine (0,n) – Caratteristica (0,n)

Una riga d'ordine può non includere alcuna personalizzazione o averne molteplici; una specifica caratteristica del menu può essere richiamata in zero o molte righe d'ordine.

Possiede: Prodotto (0,n) – Caratteristica (1,n)

Un prodotto può essere un elemento base senza varianti (0,n), ma una caratteristica inserita nel database deve necessariamente appartenere ad almeno un prodotto del menu.

Appartiene: Caratteristica (0,1) – GruppoMutuaEsclusione (2,n)

Una caratteristica può essere libera e non appartenere a nessun gruppo o appartenere al massimo a uno; un gruppo di mutua esclusione deve racchiudere almeno due caratteristiche per avere senso semantico.

Compone: Prodotto (1,n) – Ingrediente (1,n)

Relazione multi-a-molti: ogni prodotto è composto da almeno un ingrediente, e ogni ingrediente censito è associato ad almeno un prodotto.

Riceve: Ordine (1,n) – CambioStato (1,1)

Un ordine riceve almeno uno stato iniziale (inserito) e si evolve nel tempo; ogni record di cambio stato appartiene a un solo ordine.

Esegue: Personale (0,n) – CambioStato (1,1)

Un membro del personale può non aver ancora preso in carico o modificato ordini, ma ogni variazione di stato registrata deve essere firmata da un solo operatore.

2.1.5 Formalizzazione dei vincoli non esprimibili nel modello ER

Mutua esclusione applicativa tra caratteristiche dello stesso gruppo

Se un DettaglioOrdine associa delle caratteristiche, non può associare più di una caratteristica appartenente allo stesso GruppoMutuaEsclusione.

Integrità della personalizzazione

Le caratteristiche associate a un determinato DettaglioOrdine tramite la relazione Associa devono obbligatoriamente far parte del sottoinsieme di caratteristiche che il Prodotto referenziato possiede tramite la relazione Possiede.

Valori ammessi per l'attributo stato in CambioStato

L'attributo stato può assumere rigorosamente solo i valori: "inserito", "in preparazione", "pronto", "in consegna", "consegnato".

Propedeuticità temporale e sequenzialità degli stati dell'ordine

Un ordine non può retrocedere di stato. Inoltre, i successivi inserimenti in CambioStato per lo stesso ordine devono avere timestamp strettamente crescenti.

Controllo sull'orario di consegna richiesto

L'attributo orarioConsegnaRichiesto in Ordine deve essere maggiore o uguale al timeInserimento aumentato del tempoStimato calcolato, e non può superare l'orario di chiusura dell'attività.

Calcolo derivato del prezzo totale dell'ordine

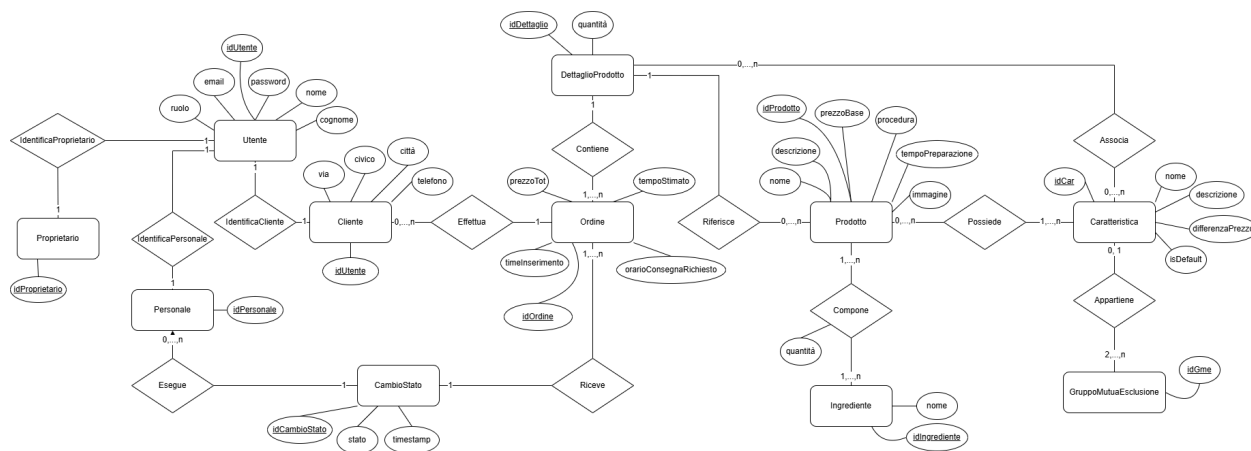
L'attributo prezzoTotale in Ordine non è un valore atomico indipendente, ma deve corrispondere esattamente alla sommatoria dei prezzi calcolati in ogni DettaglioOrdine.

Esclusività degli attributi di default

All'interno di un GruppoMutuaEsclusione, solo una caratteristica può avere l'attributo isDefault = true.

2.2 Ristrutturazione dello schema E-R

In questa fase sono state adottate scelte di design e architetturali finalizzate alla normalizzazione del database e all'ottimizzazione delle prestazioni, con particolare attenzione alle query più complesse e frequenti nel dominio applicativo.



L'immagine originale è disponibile come allegato.

2.2.1 Analisi delle ridondanze

Nel modello concettuale iniziale erano stati individuati due attributi potenzialmente ridondanti all'interno dell'entità Ordine: prezzoTotale e tempoStimato. Entrambi sono valori derivati: il primo

dalla somma dei prezzi dei prodotti e delle caratteristiche, il secondo dai tempi di preparazione dei singoli piatti. Si è deciso di mantenere questi due attributi come dati persistenti poiché in un'attività commerciale, i listini prezzi e i tempi delle ricette variano nel tempo. Memorizzare il valore finale calcolato al momento dell'inserimento blocca lo storico dell'ordine. Ricalcolarli al tramite operazioni di JOIN sulle tabelle del menu rischierebbe di alterare retroattivamente l'importo di ordini passati in caso di futuri aggiornamenti ai prezzi dei prodotti, compromettendo l'integrità dei dati storici.

2.2.2 Eliminazione della gerarchia Utente

Il modello concettuale presentava una gerarchia in cui Utente si specializzava in Proprietario, Personale e Cliente. È stata applicata la tecnica del Partizionamento verticale completo. L'entità padre Utente viene mantenuta per centralizzare i dati di autenticazione condivisi. Da essa si diramano tre entità figlie, ciascuna collegata a Utente tramite una relazione uno-a-uno. Questa architettura garantisce il massimo rigore nell'integrità referenziale del database. La relazione che traccia gli avanzamenti operativi collega l'entità CambioStato esclusivamente all'entità Personale. A livello fisico, questo impedisce a monte che l'ID di un cliente possa mai essere inserito come autore di un aggiornamento di stato, demandando la sicurezza della logica di business alla robustezza dello schema relazionale stesso. L'attributo ruolo è stato conservato in Utente come ridondanza controllata, utile per instradare rapidamente le autorizzazioni lato backend.

2.2.3 Introduzione di identificatori surrogati e ridenominazione

Nel modello concettuale, entità come Prodotto o Caratteristica utilizzavano stringhe come identificatori primari, mentre altre erano modellate come entità deboli. Tutte le entità sono state dotate di un identificatore surrogato numerico auto-incrementante. L'entità debole DettaglioOrdine è stata inoltre rinominata in DettaglioProdotto per riflettere meglio la semantica di istanza personalizzata di un prodotto all'interno del carrello. L'uso di ID numerici interi ottimizza drasticamente le prestazioni del database durante le operazioni di JOIN e alleggerisce le tabelle associative.

2.3 Traduzione del modello E-R nel modello relazionale

Di seguito lo schema logico relazionale derivato dalla ristrutturazione:

- Utente (idUtente, email, password, nome, cognome, ruolo)
- Proprietario (idProprietario, idUtente)
- Personale (idPersonale, idUtente)
- Cliente (idCliente, idUtente, telefono, via, civico, città)
- Prodotto (idProdotto, nome, descrizione, prezzoBase, tempoPreparazione, procedura, immagine)
- Ingrediente (idIngrediente, nome)
- ComposizioneProdotto (idProdotto, idIngrediente, quantità)
- GruppoMutuaEsclusione (idGmc, nome, descrizione)

- Caratteristica (idCaratteristica, nome, descrizione, differenzaPrezzo, isDefault, idGmc)
- PossessoCaratteristica (idProdotto, idCaratteristica)
- Ordine (idOrdine, timeInserimento, orarioConsegnaRichiesto, tempoStimato, prezzoTot, idCliente)
- DettaglioProdotto (idDettaglio, quantita, idOrdine, idProdotto)
- AssociazioneCaratteristica (idDettaglio, idCaratteristica)
- CambioStato (idCambioStato, stato, timestamp, idOrdine, idPersonale)

3 Scelte Progettuali

Durante lo sviluppo dell'API, sono state prese diverse decisioni architetturali per garantire scalabilità, sicurezza e manutenibilità del codice

3.1 Paradigma Collection-Item

La definizione degli endpoint è stata rigorosamente progettata per aderire al paradigma Collection-Item. Ad esempio, la risorsa Prodotti viene interrogata globalmente tramite /prodotti (Collection), mentre il singolo elemento viene manipolato specificando il suo identificativo tramite /prodotti/{id} (Item). Questo garantisce un'interfaccia predicibile e standardizzata.

3.2 Design Pattern DAO

L'accesso ai dati è stato separato dalla logica di presentazione REST utilizzando il pattern DAO. Classi specifiche incapsulano le query SQL, mantenendo i Resource REST puliti e focalizzati solo sulla gestione delle richieste e risposte HTTP.

3.3 Sicurezza tramite JSON Web Token

L'autenticazione è stateless. Al momento del login, il server genera un token JWT firmato tramite HMAC-SHA di io.jsonwebtoken che include l'ID dell'utente e il suo ruolo. Il filtro globale JwtAuthFilter intercetta le richieste dirette agli endpoint protetti marcati con l'annotazione custom @Secured, validando il token e garantendo l'accesso solo agli utenti autorizzati.

3.4 Gestione delle Transazioni

Per le operazioni complesse che coinvolgono più scritture sul database, come la creazione di un nuovo ordine, è stato disabilitato l'autocommit. Questo garantisce che in caso di errore durante l'inserimento dei dettagli o il calcolo dei prezzi, venga eseguito un rollback(), mantenendo la consistenza dei dati.

3.5 Gestione Globale delle Eccezioni

È stato implementato un GlobalExceptionHandler per intercettare qualsiasi errore non gestito a livello di codice. Questo garantisce che, in caso di errori critici, il server non restituisca mai lo stack trace o pagine HTML standard di Tomcat, ma fornisca sempre una risposta JSON pulita con codice di stato 500 Internal Server Error, migliorando l'affidabilità per il client.

3.6 Cross-Origin Resource Sharing

Per permettere il consumo dell'API da parte di Web Application ospitate su domini differenti, è stato implementato il CorsFilter, che inietta dinamicamente gli header Access-Control-Allow-Origin e definisce i metodi consentiti in ogni singola risposta HTTP.

4 Dipendenze Software

L'applicazione è stata sviluppata riducendo al minimo le dipendenze esterne, privilegiando l'utilizzo di specifiche standard

- Jakarta EE 10/11 (JAX-RS) core per la realizzazione dei servizi RESTful.
- Server Web Apache Tomcat.
- Driver JDBC MySQL
- JWT utilizzata per la generazione, firma crittografica e parsing dei JSON Web Token.

5 Funzionalità Realizzate

Il sistema implementa in modo completo i seguenti flussi operativi

5.1 Gestione Autenticazione

Login, registrazione e gestione dei permessi divisi su tre livelli gerarchici: proprietario, personale, e cliente.

5.2 Gestione Catalogo

Inserimento, modifica ed eliminazione di prodotti, categorie e ingredienti. È stata implementata in modo approfondito la gestione delle caratteristiche aggiuntive dei prodotti, supportando i gruppi di mutua esclusione e varianti di prezzo dinamiche.

5.3 Gestione Ordini

L'inserimento di un ordine innesca il calcolo automatico dei tempi di preparazione totali e del prezzo finale. È possibile monitorare e aggiornare gli stati dell'ordine tramite una macchina a stati finiti validata nel backend.

1.5.4 Data Masking

Per ragioni di segretezza, l'endpoint GET /prodotti/{idProdotto}/ingredienti filtra dinamicamente il payload restituito. Se la richiesta proviene da un cliente generico, le dosi degli ingredienti e le procedure di cucina vengono oscurate, venendo mostrate esclusivamente alle richieste autenticate come personale o proprietario.

6 Funzionalità Extra e Opzionali Realizzate

L'analisi dei requisiti di base forniti in specifica ha delineato un sistema funzionale per la consultazione e l'ordinazione. Tuttavia, per rendere l'applicativo una soluzione software

completa, autonoma e aderente a scenari di business reali, si è deciso di implementare un ampio set di funzionalità aggiuntive.

Queste estensioni permettono l'implementazione di una piattaforma gestionale dinamica coprendo l'intero ciclo di vita delle risorse. Di seguito vengono descritte nel dettaglio le funzionalità extra inserite e le motivazioni architetturali e di business che ne hanno guidato lo sviluppo.

6.1 Gestione Dinamica del Catalogo

La specifica richiedeva unicamente la lettura del menu e la ricerca. Sono state aggiunte tutte le operazioni di scrittura necessarie per rendere l'amministratore autonomo

POST /prodotti

PUT /prodotti/{id}

GET /prodotti/categorie e il filtro categoria in query string.

In un caso d'uso reale, il menu di un ristorante è in continua evoluzione. Permettere l'inserimento e l'aggiornamento dei prodotti tramite API evita la necessità di intervenire manualmente sul database, fornendo il backend necessario per una potenziale dashboard amministrativa frontend. L'estrazione dinamica delle categorie permette inoltre di costruire interfacce client auto aggiornanti.

6.2 Dizionario Globale degli Ingredienti

Oltre all'estrazione degli ingredienti per un singolo prodotto, è stata sviluppata una gestione globale del magazzino logico

GET /ingredienti

POST /ingredienti

DELETE /ingredienti/{id}

Disaccoppiare gli ingredienti dai singoli prodotti permette di normalizzare il database. Invece di avere stringhe ripetute, il sistema mantiene un dizionario unico, facilitando future implementazioni come la gestione delle scorte o la tracciabilità degli allergeni a livello di sistema.

6.3 Gruppi di Mutua Esclusione e Caratteristiche

La specifica richiedeva l'eliminazione di una caratteristica da un prodotto. L'implementazione ha esteso questo concetto, creando ulteriori funzionalità.

CRUD completo su /caratteristiche

CRUD completo su /caratteristiche/gruppi

POST /prodotti/{idProdotto}/caratteristiche.

6.4 Registrazione Utenti e Gestione Profilo

La specifica base partiva dal presupposto di avere utenti già esistenti (Login/Logout).

POST /auth/register

PUT /clienti/{idUtente}

Per rendere la piattaforma utilizzabile, il sistema di registrazione è fondamentale. L'endpoint di aggiornamento permette inoltre ai clienti di mantenere aggiornati i propri dati sensibili e di spedizione per le consegne a domicilio.

6.5 Gestione Dinamica del Personale

Il sistema di autenticazione distingue i ruoli, ma serviva un modo per amministrare questi permessi.

GET /personale

POST /personale/{idUtente}

DELETE /personale/{id}

Il Proprietario deve poter gestire i propri dipendenti in autonomia.

6.6 Data Masking e Sicurezza

Mentre la specifica chiedeva la "Lista degli ingredienti necessari alla preparazione di un prodotto", si è ritenuto essenziale applicare una logica di protezione del dato

GET /prodotti/{idProdotto}/ingredienti si comporta in maniera differente in base all'utente che effettua la richiesta.

Se un endpoint è pubblico, esporre le dosi esatte e la procedura di preparazione equivale a esporre la ricetta. L'API è stata progettata in modo smart, quindi se la richiesta è anonima o di un cliente, restituisce solo i nomi degli ingredienti; se la richiesta è autenticata da un membro

del Personale, il payload JSON si espande includendo le quantità e le procedure destinate alla cucina.

7 Specifica dell'API

L'architettura RESTful di Foody è stata divisa in aree logiche basate sulle risorse principali del dominio.

Di seguito viene fornito un quadro riassuntivo di tutti gli endpoint implementati. Per la specifica tecnica completa e dettagliata, comprensiva della struttura esatta dei payload JSON, delle query string, degli header di autenticazione e di tutti i codici di stato HTTP gestiti, si rimanda alla lettura del file `openapi.yaml` oppure `openapi.md` allegati al presente progetto.

7.1 Autenticazione

Gestione dell'accesso e della registrazione degli utenti al sistema.

Metodo	Endpoint	Descrizione
POST	/auth/login	Autentica un utente tramite credenziali e restituisce il token JWT per le richieste protette.
POST	/auth/register	Crea un nuovo account cliente nel sistema.

7.2 Catalogo Prodotti

Gestione del menu, delle categorie e della composizione dei piatti.

Metodo	Endpoint	Descrizione
GET	/prodotti	Restituisce il catalogo prodotti. Supporta filtri opzionali in query string (categoria, nome, prezzo).

POST	/prodotti	Aggiunge un nuovo prodotto al menu.
PUT	/prodotti/{id}	Aggiorna le informazioni di un prodotto esistente.
GET	/prodotti/categorie	Restituisce l'elenco delle categorie uniche attualmente a menu.
GET	/prodotti/{idProdotto}/ingredienti	Elenca gli ingredienti di un prodotto (mascherando le quantità se l'utente non è autorizzato).
POST	/prodotti/{idProdotto}/caratteristiche	Assegna una caratteristica (variante/opzione) a uno specifico prodotto.
DELETE	/prodotti/{idProdotto}/caratteristiche/{idCaratteristica}	Rimuove il collegamento tra una caratteristica e un prodotto.

7.3 Ingredienti

Gestione del dizionario globale degli ingredienti a disposizione della cucina.

Metodo	Endpoint	Descrizione
GET	/ingredienti	Elenco di tutti gli ingredienti registrati nel sistema.
POST	/ingredienti	Crea un nuovo ingrediente nel dizionario globale.
DELETE	/ingredienti/{id}	Rimuove un ingrediente dal sistema (se non in uso in alcuna ricetta).

7.4 Caratteristiche e Gruppi di Mutua Esclusione

Gestione delle varianti applicabili ai prodotti.

Metodo	Endpoint	Descrizione
GET	/caratteristiche	Elenco globale di tutte le caratteristiche definibili.
POST	/caratteristiche	Crea una nuova caratteristica applicabile ai prodotti.
PUT	/caratteristiche/{idCaratteristica}	Aggiorna i dati (es. variazione di prezzo) di una caratteristica.
DELETE	/caratteristiche/{idCaratteristica}	Elimina una caratteristica dal sistema.

GET	/caratteristiche/gruppi	Elenco di tutti i gruppi di mutua esclusione.
POST	/caratteristiche/gruppi	Crea un nuovo gruppo di mutua esclusione.
PUT	/caratteristiche/gruppi/{idGmc}	Aggiorna un gruppo esistente.
DELETE	/caratteristiche/gruppi/{idGmc}	Elimina un gruppo di mutua esclusione.

7.5 Ordini

Gestione del ciclo di vita delle ordinazioni, dal carrello alla consegna.

Metodo	Endpoint	Descrizione
GET	/ordini	Elenco generale degli ordini (filtrabile per stato e data).
POST	/ordini	Crea un nuovo ordine completo (ricalcolando automaticamente totale e tempistiche).
DELETE	/ordini/{idOrdine}	Annulla un ordine specifico.
GET	/ordini/{idOrdine}/dettagli	Elenca i prodotti (e relative caratteristiche scelte) inseriti in un ordine.

POST	/ordini/{idOrdine}/dettagli	Inserisce un ulteriore prodotto all'interno di un ordine esistente.
GET	/ordini/{idOrdine}/tempo-stimato	Restituisce il tempo stimato calcolato per la preparazione e consegna.
GET	/ordini/{idOrdine}/prezzo-totale	Calcola e restituisce il prezzo totale aggiornato dell'ordine.
POST	/ordini/{idOrdine}/stati	Aggiorna lo stato di avanzamento dell'ordine (es. da "pronto" a "in consegna").
GET	/ordini/{idOrdine}/operatori	Elenca gli operatori del personale coinvolti nella gestione dell'ordine.

7.6 Personale

Gestione dei dipendenti e degli accessi privilegiati.

Metodo	Endpoint	Descrizione
GET	/personale	Ottiene la lista completa dei membri del personale.
POST	/personale/{idUserente}	Promuove un utente cliente standard a membro del personale.
DELETE	/personale/{id}	Rimuove un utente dai privilegi del personale (licenziamento).

7.7 Clienti

Gestione dei profili utente standard e dello storico personale.

Metodo	Endpoint	Descrizione
PUT	/clienti/{idUserente}	Aggiorna i dati del profilo di uno specifico cliente (es. indirizzo, telefono).
GET	/clienti/{idUserente}/ordini	Restituisce lo storico completo degli ordini effettuati da un singolo cliente.